BÀI TẬP TÌM KIẾM VÀ HEURISTIC TRONG AI

Họ và tên	Hồ Hoàng Khang
MSSV / Lớp	3123410150

Mục tiêu học tập

• Mô hình hóa bài toán tìm kiếm trong AI.

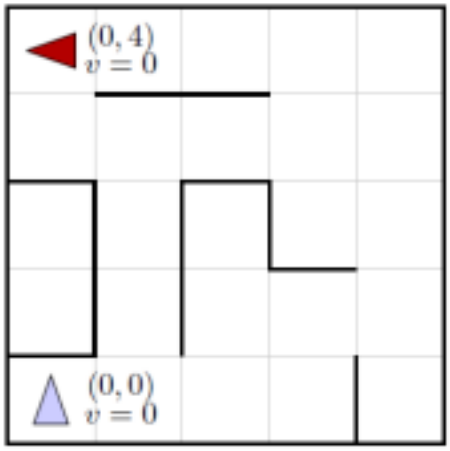
• Vận dụng BFS, UCS, Greedy và A*.

• Phân tích và đánh giá heuristic.

• Phân biệt heuristic admissible và consistent.

• So sánh tính đầy đủ, tối ưu và hiệu quả của các thuật toán tìm kiếm.

Câu 1. Tìm kiếm và heuristic



Một tác tử dạng xe cần thoát khỏi mê cung như hình trên. Tác tử có hướng và tại mọi thời điểm luôn quay về một trong bốn hướng d ∈ {Bắc, Nam, Đông, Tây}. Trong mỗi bước, tác tử có thể tiến về phía trước với vận tốc có thể điều chỉnh hoặc thực hiện thao tác rẽ.

Các hành động di chuyển gồm tăng tốc, giữ tốc độ và giảm tốc. Sau hành động, tác tử di chuyển một số ô bằng vận tốc mới v'. Gọi v là vận tốc hiện tại và v' là vận tốc sau điều chỉnh:

• Tăng tốc (Faster): v' = v + 1

• Giảm tốc (Slower): v' = v - 1

• Giữ tốc độ (Maintain): v' = v

Các hành động rẽ gồm trái và phải, làm thay đổi hướng 90°. Chỉ được phép rẽ khi vận tốc bằng 0; sau khi rẽ vận tốc vẫn bằng 0.

• Rẽ trái: quay 90° ngược chiều kim đồng hồ.

• Rẽ phải: quay 90° theo chiều kim đồng hồ.

Ví dụ: nếu tác tử ở (0,0), quay về Bắc, v = 0 và cần đến (2,0), quay về Đông, v = 0 thì một chuỗi hành động hợp lệ là: rẽ phải, tăng tốc, giữ tốc độ, giảm tốc.

Các hành động không hợp lệ gồm:

• Hành động làm tác tử va vào tường.

• Làm $v < 0$ hoặc $v > V_{max}$.

• Giữ tốc độ khi $v = 0$.

• Rẽ khi $v \neq 0$.

Mục tiêu là tìm kế hoạch đưa xe dừng hẳn ($v = 0$) tại ô thoát, đúng hướng đích, với số hành động ít nhất. Chi phí đường đi được tính bằng số hành động đã thực hiện.

a) Giả sử tác tử đi theo nhánh ngoài cùng bên trái (đi qua ô (1,2)), từ trạng thái bắt đầu (0,0), hướng Bắc đến trạng thái đích (0,4), hướng Tây. Hãy viết chuỗi hành động ngắn nhất.

Chuỗi hành động ngắn nhất: Rẽ phải, Tăng tốc, Giữ tốc độ, Giảm tốc, Rẽ trái, Tăng tốc, Giữ tốc độ, Giảm tốc, Rẽ trái.

b) Nếu lưới có kích thước $M \times N$ và vận tốc tối đa là V_{max}, kích thước không gian trạng thái là bao nhiêu? Giả sử mọi cấu hình đều có thể đạt được từ trạng thái bắt đầu.

Kích thước không gian trạng thái: $M \times N \times 4 \times (V_{max} + 1)$.

c) Một "nút con" của trạng thái s là trạng thái s' có thể đạt được bằng một hành động hợp lệ từ s. Có thể tồn tại trạng thái không có nút con hay không? Nếu có, hãy cho ví dụ; nếu không, hãy giải thích.

☑ Có ☐ Không

Có. Ví dụ: Trạng thái xe ở vị trí ô vuông bị kẹt giữa 4 bức tường xung quanh hoặc ở ô góc với vận tốc $v > 0$ mà hướng thẳng vào tường (mọi hành động di chuyển hay rẽ đều sẽ va vào tường ngay ở bước tiếp theo).

d) Hệ số phân nhánh lớn nhất của bài toán là bao nhiêu? Hãy cho một trạng thái ví dụ (x, y, hướng, vận tốc) đạt hệ số phân nhánh này và liệt kê các hành động có thể thực hiện.

Hệ số phân nhánh lớn nhất là 3.
Trạng thái ví dụ: (1, 1, Bắc, 1) ở giữa không gian trống.
Các hành động có thể thực hiện: Tăng tốc (Faster), Giữ tốc độ (Maintain), Giảm tốc (Slower).

e) Khoảng cách Manhattan từ vị trí hiện tại của tác tử đến vị trí cửa ra có phải là heuristic chấp nhận được không? Nếu không, hãy cho một trạng thái phản ví dụ, nêu giá trị heuristic và chi phí thực tế tối ưu đến đích.

☐ Có ☑ Không

Không.
Phản ví dụ: Xe ở ô (0,0), cách cửa ra (0,4) khoảng cách Manhattan là 4. Vì mỗi bước xe có thể di chuyển nhiều ô (khi $v > 1$), ví dụ nếu v tăng lên 2 hay 3, xe chỉ mất 3 hành động để tới cửa ra. Do đó $h(s) = 4 > h^*(s) = 3$ (đánh giá cao hơn chi phí thực tế tối ưu).

f) Heuristic h = khoảng cách Manhattan / V_{max} có phải là admissible không? Giải thích. Nếu không, hãy cho phản ví dụ tương tự câu (e).

☑ Có ☐ Không

Có. Vì vận tốc tối đa của xe trong một bước di chuyển là Vmax ô, nên số hành động di chuyển tối thiểu cần thiết để đi hết khoảng cách Manhattan L luôn ít nhất là ceil(L / Vmax). Do đó, h(s) = L / Vmax ≤ ceil(L / Vmax) ≤ h*(s), không bao giờ vượt quá chi phí thực tế tối ưu.

g) Nếu dùng heuristic không admissible trong A* Tree Search, tính đầy đủ của thuật toán có thể thay đổi không? Giả sử đồ thị hữu hạn và heuristic không âm.

☐ Có ☑ Không

Không. Vì đồ thị hữu hạn và chi phí mỗi bước không âm, thuật toán A* vẫn duyệt theo thứ tự ưu tiên g + h và cuối cùng sẽ tìm thấy mục tiêu nếu đường đi tới mục tiêu tồn tại.

h) Nếu dùng heuristic không admissible trong A* Tree Search, tính tối ưu của thuật toán có thể thay đổi không? Giả sử đồ thị hữu hạn và heuristic không âm.

☑ Có ☐ Không

Có. Nếu heuristic không admissible (đánh giá quá cao chi phí ở một số đường đi tối ưu), A* Tree Search có thể bỏ qua nút thuộc đường đi tối ưu đó và chấp nhận một đường đi không tối ưu khác đến đích trước.

i) Những lý do nào sau đây có thể hợp lý khi dùng heuristic không admissible thay vì admissible? Chọn tất cả phương án phù hợp.

☑ Heuristic không admissible có thể dễ tính hơn, giúp giảm thời gian tính heuristic.

☑ Heuristic không admissible có thể gần chi phí thực hơn (dù đôi khi đánh giá cao), nhờ đó có thể mở ít nút hơn.

☐ Heuristic không admissible vẫn luôn tìm được đường đi tối ưu khi mọi chi phí thực không âm.

☐ Heuristic không admissible có thể được dùng để làm thuật toán hoàn toàn không xét một phần của đồ thị.

Câu 2. Các nút tìm kiếm

Xét tree search (không dùng tập explored) cho một bài toán tìm kiếm bất kỳ có hệ số phân nhánh tối đa b. Mỗi nút tìm kiếm n có chi phí tích lũy g(n), heuristic admissible h(n) và độ sâu d(n). Gọi n_c là nút đích có chi phí nhỏ nhất và n_s là nút đích nông nhất.

Với mỗi trường hợp, hãy viết một bất đẳng thức mô tả tập các nút được mở rộng trước khi thuật toán kết thúc. Không cần xử lý trường hợp hòa (tie). Nếu không có nút nào thỏa, ghi "không có".

a) Breadth-First Search (BFS).

Bất đẳng thức: d(n) < d(n_s)

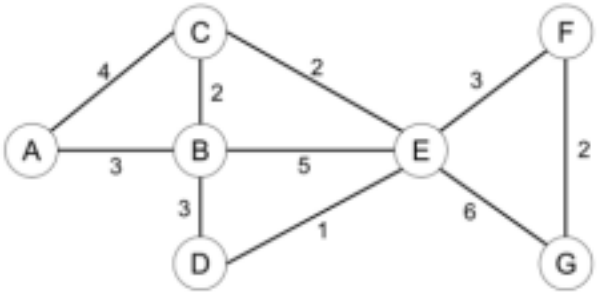
b) Uniform-Cost Search (UCS).

Bất đẳng thức: g(n) < g(n_c)

c) A* Search, giả sử h là heuristic nhất quán.

Bất đẳng thức: g(n) + h(n) < g(n_c)

Câu 3. Tìm kiếm trên đồ thị



Node	h_1	h_2
A	12	11
B	6	7
C	9	6
D	3	4
E	3	5
F	2	1
G	0	0

Nút	h1	h2
A	12	11
B	6	7
C	9	6
D	3	4
E	3	5
F	2	1
G	0	0

Xét đồ thị vô hướng như hình. A là trạng thái bắt đầu và G là trạng thái đích. Chi phí của mỗi cạnh được ghi trên đồ thị. Bảng trên cung cấp hai heuristic h1 và h2. Khi có nhiều nút cùng ưu tiên, kết quả có thể phụ thuộc quy tắc phá hòa (tie-breaking).

a) Với mỗi chiến lược tìm kiếm đồ thị dưới đây, hãy đánh dấu X vào mọi đường đi mà thuật toán có thể trả về. Nếu kết quả không nằm trong các đường đi liệt kê, ghi đường đi đúng ở cột "Khác".

Thuật toán	A-C-E-G	A-C-E-F-G	A-B-D-E-F-G	Khác
UCS	☐	☑	☐	☐
Greedy với h1	☐	☐	☑	☐
Greedy với h2	☑	☐	☐	☐

A* với h1	☐	☑	☐	☐
A* với h2	☐	☑	☐	☐

b) Chi phí của đường đi tối ưu từ A đến G theo UCS là bao nhiêu?

Đáp án: 11 (theo đường đi A-C-E-F-G: 4 + 2 + 3 + 2 = 11)

c) Heuristic h1 có admissible không? Có consistent không?

Admissible: ☐ Có ☑ Không Consistent: ☐ Có ☑ Không

d) Heuristic h2 có admissible không? Có consistent không?

Admissible: ☑ Có ☐ Không Consistent: ☑ Có ☐ Không

Câu 4. Trắc nghiệm và trả lời ngắn về tìm kiếm

a) Đánh dấu Đúng/Sai. Giả sử mọi hành động có chi phí ít nhất ε, với ε > 0, và mọi heuristic được dùng đều consistent.

Depth-First Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.

☐ Đúng ☑ Sai

Breadth-First Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.

☑ Đúng ☐ Sai

Iterative Deepening Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete. ☑ Đúng ☐ Sai

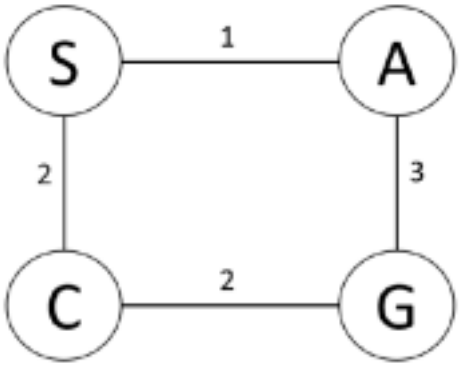
Với mọi đồ thị không có chu trình, Graph Search luôn có frontier lớn hơn Tree Search.

☐ Đúng ☑ Sai

Iterative Deepening Graph Search có độ phức tạp thời gian của BFS và độ phức tạp bộ nhớ của DFS. ☐ Đúng ☑ Sai

Nếu h1(s) và h2(s) đều là heuristic consistent thì min(h1(s), h2(s)) cũng nhất thiết consistent. ☑ Đúng ☐ Sai

b) Xét đồ thị trạng thái dưới đây, S là bắt đầu và G là đích. Hãy điền các giá trị heuristic sao cho heuristic admissible nhưng không consistent.



Heuristic Function	
State	$h(s)$
S	
A	
C	
G	0

Trạng thái	h(s)
S	4
A	3
C	1
G	0

--- Hết —